

xarray

Python para ciencias de la atmósfera

Lucía Pedraza Díaz

Datos

- Datos: ERA5, medias mensuales sobre México
- Los dos archivos vienen de **ERA5**, el reanálisis del Centro Europeo de Pronósticos a Mediano Plazo (ECMWF), distribuido por el Copernicus Climate Data Store. Un reanálisis combina un modelo numérico del tiempo con observaciones históricas para reconstruir el estado de la atmósfera de forma consistente en todo el planeta. No son observaciones directas, sino la mejor estimación de un modelo restringido por observaciones.
- El producto es *ERA5 monthly averaged data on single levels*: variables de superficie (un solo nivel), promediadas por mes.
- Contenido de cada archivo:
- **data_stream-moda_stepType-avgad.nc**: variable t2m, temperatura del aire a 2 m sobre la superficie, en kelvin.
- **data_stream-moda_stepType-avgad.nc**: variable tp, precipitación total. Unidades: **m/día**. Cada valor es la precipitación diaria promedio de ese mes.

```
xarray_practica
xarray_practica code ~/.cdsapirc
xarray_practica
```

```
url: https://cds.climate.copernicus.eu/api  
key: ●●●●●●●●●●●●●●●●●●●●
```



```
xarray_practica pip install cdsapi
```

```
import cdsapi

c = cdsapi.Client()
c.retrieve(
    "reanalysis-era5-single-levels-monthly-means",
    {
        "product_type": ["monthly_averaged_reanalysis"],
        "variable": ["2m_temperature", "total_precipitation"],
        "year": [str(y) for y in range(1991, 2025)],
        "month": [f"{m:02d}" for m in range(1, 13)],
        "time": ["00:00"],
        "area": [33, -118, 14, -86], # N, W, S, E
        "data_format": "netcdf",
        "download_format": "unarchived",
    },
    "datos/era5_mex_mensual.nc",
)
```

- Lo principal que ganas es reproducibilidad: la petición queda en código, así que la puedes repetir o modificar sin rehacer clics, y se puede correr en el cluster sin navegador. Para un archivo suelto la diferencia es mínima; se nota cuando bajas muchos o los mismos datos cada cierto tiempo.

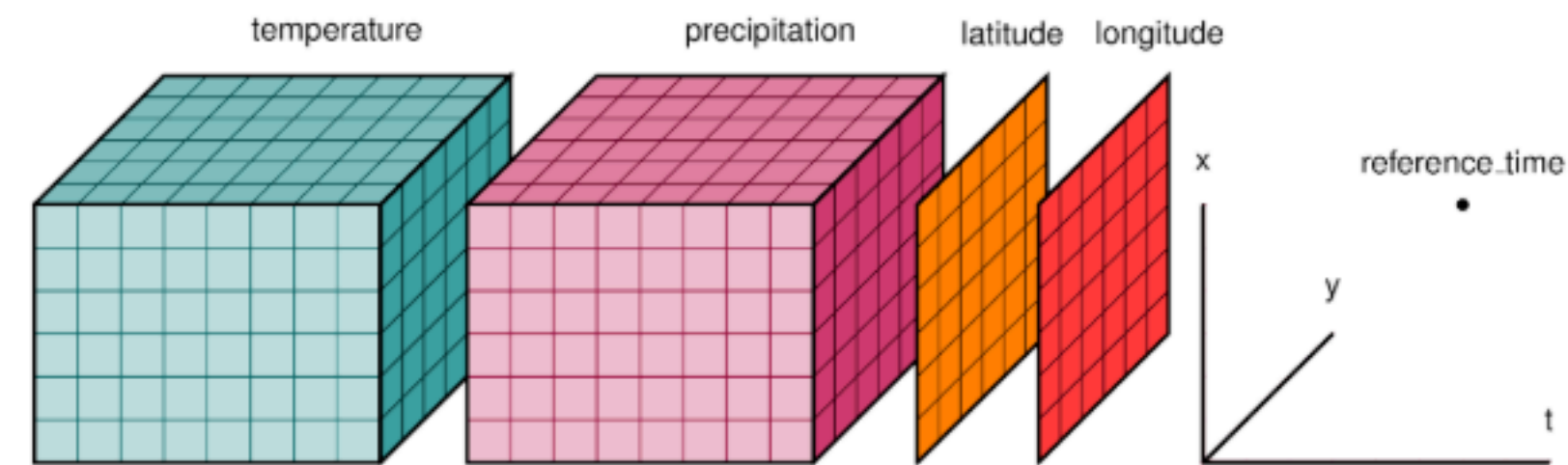
<https://xarray.dev/>

Xarray

N-D labeled arrays and datasets in Python

Xarray is an open source project and Python package that introduces labels in the form of dimensions, coordinates, and attributes on top of raw NumPy-like arrays, which allows for more intuitive, more concise, and less error-prone user experience.

Xarray includes a large and growing library of domain-agnostic functions for advanced analytics and visualization with these data structures.



[Get Started](#)

[Why Xarray?](#)

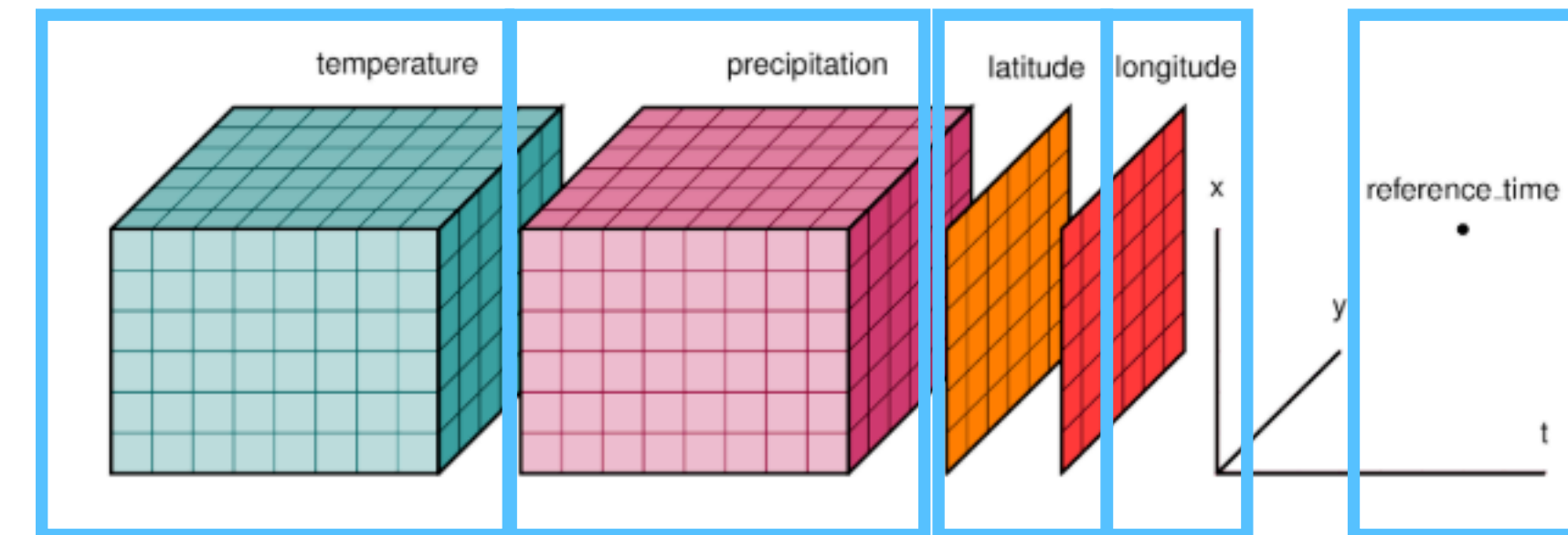
Xarray

- Los datos atmosféricos casi siempre son arreglos de varias dimensiones: temperatura en cada tiempo, latitud y longitud, a veces también en cada nivel vertical. Con numpy puro, eso es un arreglo de forma (408, 77, 129) y nada más. numpy no sabe que el primer eje es tiempo.
- xarray resuelve eso **etiquetando** los datos, de dos maneras:
 - Nombres de dimensión: cada eje tiene nombre (valid_time, latitude, longitude). Por eso podemos escribir `.mean('valid_time')` en lugar de `.mean(axis=0)`.
 - **Coordenadas:** cada eje tiene sus valores asociados (las fechas, las latitudes, las longitudes). Por eso podemos seleccionar por valor, por ejemplo "dame 20°N" o "dame mayo de 2023", en lugar de por posición.
- Además guarda **atributos** (unidades, descripción) junto a los datos,

Los objetos de xarray

Objetos principales

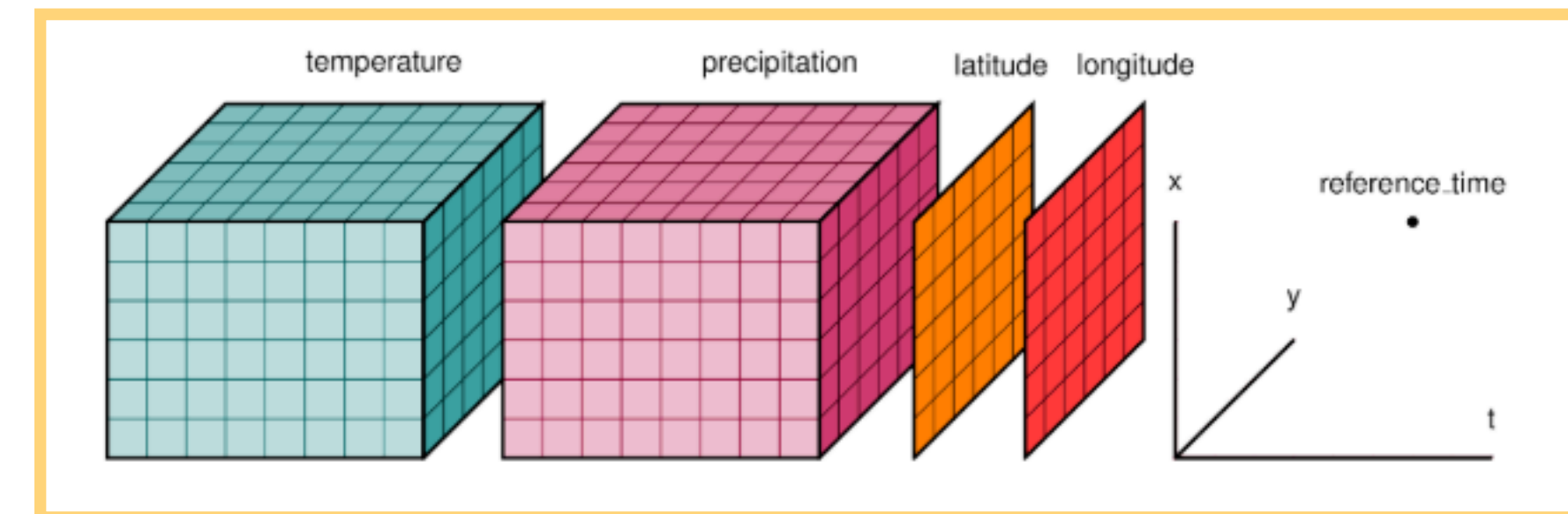
- `dataArray`: un solo arreglo etiquetado. Tiene los valores, los nombres de sus dimensiones, sus coordenadas y sus atributos.



Los objetos de xarray

Objetos principales

- **dataArray**: un solo arreglo etiquetado. Tiene los valores, los nombres de sus dimensiones, sus coordenadas y sus atributos.
- **DataSet**: una colección de **dataArray** que comparten dimensiones, más atributos globales. Es lo que obtienes al abrir un archivo NetCDF.



Dataset

- Un Dataset en xarray es un contenedor con cuatro partes:
- Dimensiones (sizes): los ejes de los datos y su tamaño, por ejemplo: valid_time: 408, latitude: 77, longitude: 129
- Coordenadas (coords): los valores de cada eje, es decir, las fechas, las latitudes y las longitudes. Cada una es un DataArray.
- Variables de datos (data_vars): los campos en sí, como t2m o tp. Cada una es un DataArray
- Atributos (attrs): los metadatos, que pueden ser del archivo completo, de cada coordenada o de cada variable.

Abrir el archivo

```
import xarray as xr
from pathlib import Path

carpeta_trabajo = Path.home() / 'Desktop' / 'PYTHON_ENES_C2' / 'xarray_practica'
datos = carpeta_trabajo / 'datos' / 'era5_mex'

ds_t = xr.open_dataset(datos / 'data_stream-moda_stepType-avgua.nc')
```

print(ds) da un
resumen completo

```
<xarray.Dataset> Size: 16MB
Dimensions:      (valid_time: 408, latitude: 77, longitude: 129)
Coordinates:
  number         int64 8B ...
  * valid_time    (valid_time) datetime64[ns] 3kB 1991-01-01 ... 2024-12-01
  * latitude      (latitude) float64 616B 33.0 32.75 32.5 ... 14.5 14.25 14.0
  * longitude     (longitude) float64 1kB -118.0 -117.8 -117.5 ... -86.25 -86.0
  expver         (valid_time) <U4 7kB ...
Data variables:
  t2m            (valid_time, latitude, longitude) float32 16MB ...
Attributes:
  GRIB_centre:      ecmf
  GRIB_centreDescription: European Centre for Medium-Range Weather Forecasts
  GRIB_subCentre:    0
  Conventions:      CF-1.7
  institution:      European Centre for Medium-Range Weather Forecasts
  history:          2026-10-05T19:33 GRIB to CDM+CF via cfgrib-0.9.1...
```

sizes, coords y data_vars

```
print(ds_t.sizes)
```

```
Frozen({'valid_time': 408, 'latitude': 77, 'longitude': 129})
```

```
print(ds_t.coords)
```

```
Coordinates:
  number      int64 8B ...
* valid_time  (valid_time) datetime64[ns] 3kB 1991-01-01 ... 2024-12-01
* latitude    (latitude) float64 616B 33.0 32.75 32.5 ... 14.5 14.25 14.0
* longitude   (longitude) float64 1kB -118.0 -117.8 -117.5 ... -86.25 -86.0
  expver      (valid_time) <U4 7kB ...
```

```
print(ds_t.data_vars)
```

```
Data variables:
  t2m          (valid_time, latitude, longitude) float32 16MB ...
```

Acceder a una variable

```
print(ds_t['t2m'])
```

```
<xarray.DataArray 't2m' (valid_time: 408, latitude: 77, longitude: 129)> Size: 16MB
[4052664 values with dtype=float32]
Coordinates:
  number      int64 8B ...
 * valid_time  (valid_time) datetime64[ns] 3kB 1991-01-01 ... 2024-12-01
 * latitude    (latitude) float64 616B 33.0 32.75 32.5 ... 14.5 14.25 14.0
 * longitude   (longitude) float64 1kB -118.0 -117.8 -117.5 ... -86.25 -86.0
  expver      (valid_time) <U4 7kB ...
```

```
Attributes: (12/32)
  GRIB_paramId:      167
  GRIB_dataType:      an
  GRIB_numberOfPoints: 9933
  GRIB_typeOfLevel:   surface
  GRIB_stepUnits:     1
  GRIB_stepType:      avgua
  ...
  GRIB_totalNumber:   0
  GRIB_units:         K
  long_name:          2 metre temperature
  units:              K
  standard_name:      unknown
  GRIB_surface:       0.0
```

Atributos de un DataArray

```
print(ds_t['t2m'].attrs)
```

```
{'GRIB_paramId': np.int64(167), 'GRIB_dataType': 'an', 'GRIB_numberOfPoints': np.int64(9933), 'GRIB_typeOfLevel': 'surface',  
'GRIB_stepUnits': np.int64(1), 'GRIB_stepType': 'avgua', 'GRIB_gridType': 'regular_ll', 'GRIB_uvRelativeToGrid': np.int64(0), 'GRIB_NV':  
np.int64(0), 'GRIB_Nx': np.int64(129), 'GRIB_Ny': np.int64(77), 'GRIB_cfName': 'unknown', 'GRIB_cfVarName': 't2m',  
'GRIB_gridDefinitionDescription': 'Latitude/Longitude Grid', 'GRIB_iDirectionIncrementInDegrees': np.float64(0.25),  
'GRIB_iScansNegatively': np.int64(0), 'GRIB_jDirectionIncrementInDegrees': np.float64(0.25), 'GRIB_jPointsAreConsecutive': np.int64(0),  
'GRIB_jScansPositively': np.int64(0), 'GRIB_latitudeOfFirstGridPointInDegrees': np.float64(33.0),  
'GRIB_latitudeOfLastGridPointInDegrees': np.float64(14.0), 'GRIB_longitudeOfFirstGridPointInDegrees': np.float64(-118.0),  
'GRIB_longitudeOfLastGridPointInDegrees': np.float64(-86.0), 'GRIB_missingValue': np.float64(3.4028234663852886e+38), 'GRIB_name': '2  
metre temperature', 'GRIB_shortName': '2t', 'GRIB_totalNumber': np.int64(0), 'GRIB_units': 'K', 'long_name': '2 metre temperature',  
'units': 'K', 'standard_name': 'unknown', 'GRIB_surface': np.float64(0.0)}
```

```
print(ds_t['t2m'].attrs['long_name'])
```

```
2 metre temperature
```

Acceder a una coordenada

```
print(ds_t['longitude'])
```

```
<xarray.DataArray 'longitude' (longitude: 129)> Size: 1kB
array([ -118.   , -117.75, -117.5 , -117.25, -117.   , -116.75, -116.5 , -116.25,
        -116.   , -115.75, -115.5 , -115.25, -115.   , -114.75, -114.5 , -114.25,
        -114.   , -113.75, -113.5 , -113.25, -113.   , -112.75, -112.5 , -112.25,
        -112.   , -111.75, -111.5 , -111.25, -111.   , -110.75, -110.5 , -110.25,
        -110.   , -109.75, -109.5 , -109.25, -109.   , -108.75, -108.5 , -108.25,
        -108.   , -107.75, -107.5 , -107.25, -107.   , -106.75, -106.5 , -106.25,
        -106.   , -105.75, -105.5 , -105.25, -105.   , -104.75, -104.5 , -104.25,
        -104.   , -103.75, -103.5 , -103.25, -103.   , -102.75, -102.5 , -102.25,
        -102.   , -101.75, -101.5 , -101.25, -101.   , -100.75, -100.5 , -100.25,
        -100.   ,  -99.75,  -99.5 ,  -99.25,  -99.   ,  -98.75,  -98.5 ,  -98.25,
         -98.   ,  -97.75,  -97.5 ,  -97.25,  -97.   ,  -96.75,  -96.5 ,  -96.25,
         -96.   ,  -95.75,  -95.5 ,  -95.25,  -95.   ,  -94.75,  -94.5 ,  -94.25,
         -94.   ,  -93.75,  -93.5 ,  -93.25,  -93.   ,  -92.75,  -92.5 ,  -92.25,
         -92.   ,  -91.75,  -91.5 ,  -91.25,  -91.   ,  -90.75,  -90.5 ,  -90.25,
         -90.   ,  -89.75,  -89.5 ,  -89.25,  -89.   ,  -88.75,  -88.5 ,  -88.25,
         -88.   ,  -87.75,  -87.5 ,  -87.25,  -87.   ,  -86.75,  -86.5 ,  -86.25,
         -86.   ])
```

Coordinates:

```
number      int64 8B ...
* longitude  (longitude) float64 1kB -118.0 -117.8 -117.5 ... -86.25 -86.0
```

Attributes:

```
units:      degrees_east
standard_name: longitude
long_name:   longitude
```

Atributos de las coordenadas

```
print(ds_t['longitude'].attrs)
```

```
{'units': 'degrees_east', 'standard_name': 'longitude', 'long_name': 'longitude'}
```

```
print(ds_t['valid_time'])
```

```
<xarray.DataArray 'valid_time' (valid_time: 408)> Size: 3kB
array(['1991-01-01T00:00:00.000000000', '1991-02-01T00:00:00.000000000',
      '1991-03-01T00:00:00.000000000', ..., '2024-10-01T00:00:00.000000000',
      '2024-11-01T00:00:00.000000000', '2024-12-01T00:00:00.000000000'],
      dtype='datetime64[ns]')
Coordinates:
  number      int64 8B ...
  * valid_time (valid_time) datetime64[ns] 3kB 1991-01-01 ... 2024-12-01
  expver      (valid_time) <U4 7kB ...
Attributes:
  long_name:      time
  standard_name:  time
```

datetime64[ns] es un tipo de numpy.

Un datetime64 guarda la fecha como un entero: cuantas unidades han pasado desde el 1 de enero de 1970 (“época de Unix”)

[ns] indica nanosegundos

```
print(ds_t['valid_time'])
```

```
<xarray.DataArray 'valid time' (valid time: 408)> Size: 3kB
array(['1991-01-01T00:00:00.000000000', '1991-02-01T00:00:00.000000000',
      '1991-03-01T00:00:00.000000000', ..., '2024-10-01T00:00:00.000000000',
      '2024-11-01T00:00:00.000000000', '2024-12-01T00:00:00.000000000'],
      dtype='datetime64[ns]')
Coordinates:
  number      int64 8B ...
  * valid_time (valid_time) datetime64[ns] 3kB 1991-01-01 ... 2024-12-01
  expver      (valid_time) <U4 7kB ...
Attributes:
  long_name:    time
  standard_name: time
```

Formato estándar ISO 8601:
año-mes-día, una _T que separa la
fecha de la hora, y luego
horas:minutos:segundos con la
fracción de segundo en
nanosegundos (por eso tantos
ceros)

```
import xarray as xr
from pathlib import Path

carpeta_trabajo = Path.home() / 'Desktop' / 'PYTHON_ENES_C2' / 'xarray_practica'
datos = carpeta_trabajo / 'datos' / 'era5_mex'

ds_t = xr.open_dataset(datos / 'data_stream-moda_stepType-avgua.nc', decode_times=False)
```

```
print(ds_t['valid_time'])
```

Fechas en crudo

```
expver      (valid_time) <U4 7kB ...
Attributes:
    long_name:      time
    standard_name:  time
    units:          seconds since 1970-01-01
    calendar:       proleptic_gregorian
```

```
print(ds_t['valid_time'].values)
```

```
[ 662688000  665366400  667785600  670464000  673056000  675734400
  678326400  681004800  683683200  686275200  688953600  691545600
  694224000  696902400  699408000  702086400  704678400  707356800
  709948800  712627200  715305600  717897600  720576000  723168000
  725846400  728524800  730944000  733622400  736214400  738892800
  741484800  744163200  746841600  749433600  752112000  754704000
  757382400  760060800  762480000  765158400  767750400  770428800]
```

```
print(ds_t['valid_time'].values)
```

```
['1991-01-01T00:00:00.000000000' '1991-02-01T00:00:00.000000000'
 '1991-03-01T00:00:00.000000000' '1991-04-01T00:00:00.000000000'
 '1991-05-01T00:00:00.000000000' '1991-06-01T00:00:00.000000000'
 '1991-07-01T00:00:00.000000000' '1991-08-01T00:00:00.000000000'
 '1991-09-01T00:00:00.000000000' '1991-10-01T00:00:00.000000000'
 '1991-11-01T00:00:00.000000000' '1991-12-01T00:00:00.000000000'
 '1992-01-01T00:00:00.000000000' '1992-02-01T00:00:00.000000000'
 '1992-03-01T00:00:00.000000000' '1992-04-01T00:00:00.000000000'
 '1992-05-01T00:00:00.000000000' '1992-06-01T00:00:00.000000000'
 '1992-07-01T00:00:00.000000000' '1992-08-01T00:00:00.000000000'
 '1992-09-01T00:00:00.000000000' '1992-10-01T00:00:00.000000000'
 '1992-11-01T00:00:00.000000000' '1992-12-01T00:00:00.000000000'
 '1993-01-01T00:00:00.000000000' '1993-02-01T00:00:00.000000000'
 '1993-03-01T00:00:00.000000000' '1993-04-01T00:00:00.000000000'
 '1993-05-01T00:00:00.000000000' '1993-06-01T00:00:00.000000000'
 '1993-07-01T00:00:00.000000000' '1993-08-01T00:00:00.000000000'
 '1993-09-01T00:00:00.000000000' '1993-10-01T00:00:00.000000000'
 '1993-11-01T00:00:00.000000000' '1993-12-01T00:00:00.000000000'
 '1994-01-01T00:00:00.000000000' '1994-02-01T00:00:00.000000000'
 '1994-03-01T00:00:00.000000000' '1994-04-01T00:00:00.000000000'
 '1994-05-01T00:00:00.000000000' '1994-06-01T00:00:00.000000000'
 '1994-07-01T00:00:00.000000000' '1994-08-01T00:00:00.000000000'
 '1994-09-01T00:00:00.000000000' '1994-10-01T00:00:00.000000000'
 '1994-11-01T00:00:00.000000000' '1994-12-01T00:00:00.000000000'
 '1995-01-01T00:00:00.000000000' '1995-02-01T00:00:00.000000000'
 '1995-03-01T00:00:00.000000000' '1995-04-01T00:00:00.000000000'
 '1995-05-01T00:00:00.000000000' '1995-06-01T00:00:00.000000000'
 '1995-07-01T00:00:00.000000000' '1995-08-01T00:00:00.000000000']
```

```
print(ds_t['valid_time'].dt.year.values)
```

dt es un atributo de DataArray de xarray. Devuelve un objeto que agrupa herramientas para fechas: years, month, days_in_month

```
print(ds_t['valid_time'].dt.month.values)
```

```
[1991 1991 1991 1991 1991 1991 1991 1991 1991 1991 1991 1991 1992 1992
1992 1992 1992 1992 1992 1992 1992 1992 1992 1992 1993 1993 1993 1993
1993 1993 1993 1993 1993 1993 1993 1993 1994 1994 1994 1994 1994 1994
1994 1994 1994 1994 1994 1994 1995 1995 1995 1995 1995 1995 1995 1995
1995 1995 1995 1995 1996 1996 1996 1996 1996 1996 1996 1996 1996 1996
1996 1996 1997 1997 1997 1997 1997 1997 1997 1997 1997 1997 1997 1997
1998 1998 1998 1998 1998 1998 1998 1998 1998 1998 1998 1998 1999 1999
1999 1999 1999 1999 1999 1999 1999 1999 1999 1999 2000 2000 2000 2000
2000 2000 2000 2000 2000 2000 2000 2000 2001 2001 2001 2001 2001 2001
2001 2001 2001 2001 2001 2001 2002 2002 2002 2002 2002 2002 2002 2002
2002 2002 2002 2002 2003 2003 2003 2003 2003 2003 2003 2003 2003 2003
2003 2003 2004 2004 2004 2004 2004 2004 2004 2004 2004 2004 2004 2004
2005 2005 2005 2005 2005 2005 2005 2005 2005 2005 2005 2005 2006 2006]
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12  1  2  3  4  5  6  7  8  9 10 11 12
 1  2  3  4  5  6  7  8  9 10 11 12  1  2  3  4  5  6  7  8  9 10 11 12
 1  2  3  4  5  6  7  8  9 10 11 12  1  2  3  4  5  6  7  8  9 10 11 12
 1  2  3  4  5  6  7  8  9 10 11 12  1  2  3  4  5  6  7  8  9 10 11 12
 1  2  3  4  5  6  7  8  9 10 11 12  1  2  3  4  5  6  7  8  9 10 11 12
 1  2  3  4  5  6  7  8  9 10 11 12  1  2  3  4  5  6  7  8  9 10 11 12
 1  2  3  4  5  6  7  8  9 10 11 12  1  2  3  4  5  6  7  8  9 10 11 12
 1  2  3  4  5  6  7  8  9 10 11 12  1  2  3  4  5  6  7  8  9 10 11 12
 1  2  3  4  5  6  7  8  9 10 11 12  1  2  3  4  5  6  7  8  9 10 11 12
 1  2  3  4  5  6  7  8  9 10 11 12  1  2  3  4  5  6  7  8  9 10 11 12
 1  2  3  4  5  6  7  8  9 10 11 12  1  2  3  4  5  6  7  8  9 10 11 12
 1  2  3  4  5  6  7  8  9 10 11 12  1  2  3  4  5  6  7  8  9 10 11 12]
```

```
print(ds_t['valid_time'].dt.days_in_month.values)
```

```
[31 28 31 30 31 30 31 31 30 31 30 31 31 29 31 30 31 30 31 31 30 31 30 31
 31 28 31 30 31 30 31 31 30 31 30 31 31 28 31 30 31 30 31 31 30 31 30 31
 31 28 31 30 31 30 31 31 30 31 30 31 31 29 31 30 31 30 31 31 30 31 30 31
 31 28 31 30 31 30 31 31 30 31 30 31 31 28 31 30 31 30 31 31 30 31 30 31
 31 28 31 30 31 30 31 31 30 31 30 31 31 29 31 30 31 30 31 31 30 31 30 31
 31 28 31 30 31 30 31 31 30 31 30 31 31 28 31 30 31 30 31 31 30 31 30 31
 31 28 31 30 31 30 31 31 30 31 30 31 31 29 31 30 31 30 31 31 30 31 30 31
 31 28 31 30 31 30 31 31 30 31 30 31 31 28 31 30 31 30 31 31 30 31 30 31
 31 28 31 30 31 30 31 31 30 31 30 31 31 29 31 30 31 30 31 31 30 31 30 31]
```

```
print(ds_t['valid_time'].dt.strftime('%Y-%m'))
```

```
array(['1991-01', '1991-02', '1991-03', '1991-04', '1991-05', '1991-06',
      '1991-07', '1991-08', '1991-09', '1991-10', '1991-11', '1991-12',
      '1992-01', '1992-02', '1992-03', '1992-04', '1992-05', '1992-06',
      '1992-07', '1992-08', '1992-09', '1992-10', '1992-11', '1992-12',
      '1993-01', '1993-02', '1993-03', '1993-04', '1993-05', '1993-06',
      '1993-07', '1993-08', '1993-09', '1993-10', '1993-11', '1993-12',
      '1994-01', '1994-02', '1994-03', '1994-04', '1994-05', '1994-06',
      '1994-07', '1994-08', '1994-09', '1994-10', '1994-11', '1994-12',
      '1995-01', '1995-02', '1995-03', '1995-04', '1995-05', '1995-06',
      '1995-07', '1995-08', '1995-09', '1995-10', '1995-11', '1995-12',
      '1996-01', '1996-02', '1996-03', '1996-04', '1996-05', '1996-06',
      '1996-07', '1996-08', '1996-09', '1996-10', '1996-11', '1996-12',
      '1997-01', '1997-02', '1997-03', '1997-04', '1997-05', '1997-06',
      '1997-07', '1997-08', '1997-09', '1997-10', '1997-11', '1997-12',
```

```
print(dir(ds_t['valid_time'].dt))
```

```
['__class__', '__class_getitem__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__',
'__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__orig_bases__',
'__orig_class__', '__parameters__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__slots__', '__str__',
'__subclasshook__', '__weakref__', '_date_field', '_is_protocol', '_obj', '_tslib_round_accessor', 'calendar', 'ceil', 'date', 'day',
'dayofweek', 'dayofyear', 'days_in_month', 'daysinmonth', 'floor', 'hour', 'is_leap_year', 'is_month_end', 'is_month_start',
'is_quarter_end', 'is_quarter_start', 'is_year_end', 'is_year_start', 'isocalendar', 'microsecond', 'minute', 'month', 'nanosecond',
'quarter', 'round', 'season', 'second', 'strftime', 'time', 'week', 'weekday', 'weekofyear', 'year']
```

```
help(ds_t['valid_time'].dt.is_year_end)
```

- Nota: las expresiones son equivalentes

```
mty['valid_time.month']
mty['valid_time'].dt.month
```

Ejemplo

- Pasar la temperatura a Celcius

```
t_c = ds_t['t2m'] - 273.15  
print(t_c)
```

Ejemplo

- Revisar si la longitud viene de -180 a 180 o de 0 a 360

```
print(t_c['longitude'].values[0], t_c['longitude'].values[-1])
```

```
-118.0 -86.0
```

Ejemplo

- isel: selección por posición. Es un método de DataArray y de Dataset que selecciona datos usando la posición (el índice entero) en cada dimensión.

```
print(linea := t_c.isel(latitude=29))  
print(linea.dims)  
print(linea.shape)
```

```
array([[16.79068 , 16.818024, 16.84732 , ..., 22.444977, 22.519196,  
       22.57779 ],  
       [17.24527 , 17.27652 , 17.309723, ..., 21.667145, 21.747223,  
       21.790192],  
       [16.189117, 16.214508, 16.251617, ..., 23.448883, 23.489899,  
       23.489899],
```

```
('valid_time', 'longitude')  
(408, 129)
```

Solo la posición 29. La dimensión desaparece.

Longitud: todas

Tiempo: todos

Ejemplo

- isel: selección por posición. Es un método de DataArray y de Dataset que selecciona datos usando la posición (el índice entero) en cada dimensión.

```
print(linea := t_c.isel(latitude=29))
print(linea.dims)
print(linea.shape)
```

```
array([[16.79068 , 16.818024, 16.84732 , ..., 22.444977, 22.519196,
        22.57779 ],
       [17.24527 , 17.27652 , 17.309723, ..., 21.667145, 21.747223,
        21.790192],
       [16.189117, 16.214508, 16.251617, ..., 23.448883, 23.489899,
        23.489899],
       ('valid_time', 'longitude')
      ])
```

Solo la posición 29. La dimensión desaparece.

Longitud: todas

Tiempo: todos

```
print(linea['latitude'].values)
print(t_c['latitude'].values[29])
```

25.75

Ejemplo

.sel: selección por valor. Es un método de DataArray y de Dataset que selecciona datos usando los valores de las coordenadas (grados, fechas), no la posición.

```
print(t_c.sel(latitude=25.75))
```

```
print(t_c.sel(latitude=25.7599))
```

```
KeyError: "not all values found in index 'latitude'. Try setting the `method` keyword argument (example: method='nearest')."
```

```
mtv = t_c.sel(latitude=25.67, longitude=-100.31, method='nearest')  
print(mtv)
```

* toma el valor más cercano

```
valor = mty.sel(valid_time='2023-05-01')
print(valor)
```

```
<xarray.DataArray 't2m' (>) Size: 4B
array(24.395416, dtype=float32)
Coordinates:
  number      int64 8B ...
  valid_time   datetime64[ns] 8B 2023-05-01
  latitude     float64 8B 25.75
  longitude    float64 8B -100.2
  expver       <U4 16B ...
```

```
valor = mty.sel(valid_time='2023-05')
print(valor)
```

```
<xarray.DataArray 't2m' (valid_time: 1)> Size: 4B
array([24.395416], dtype=float32)
Coordinates:
  number      int64 8B ...
  * valid_time (valid_time) datetime64[ns] 8B 2023-05-01
  latitude     float64 8B 25.75
  longitude    float64 8B -100.2
  expver       (valid_time) <U4 16B ...
```

*expand dims para agregar una dimensión

```
valor = mty.sel(valid_time='2023-05').squeeze()
print(valor)
```

Serie de tiempo en un punto

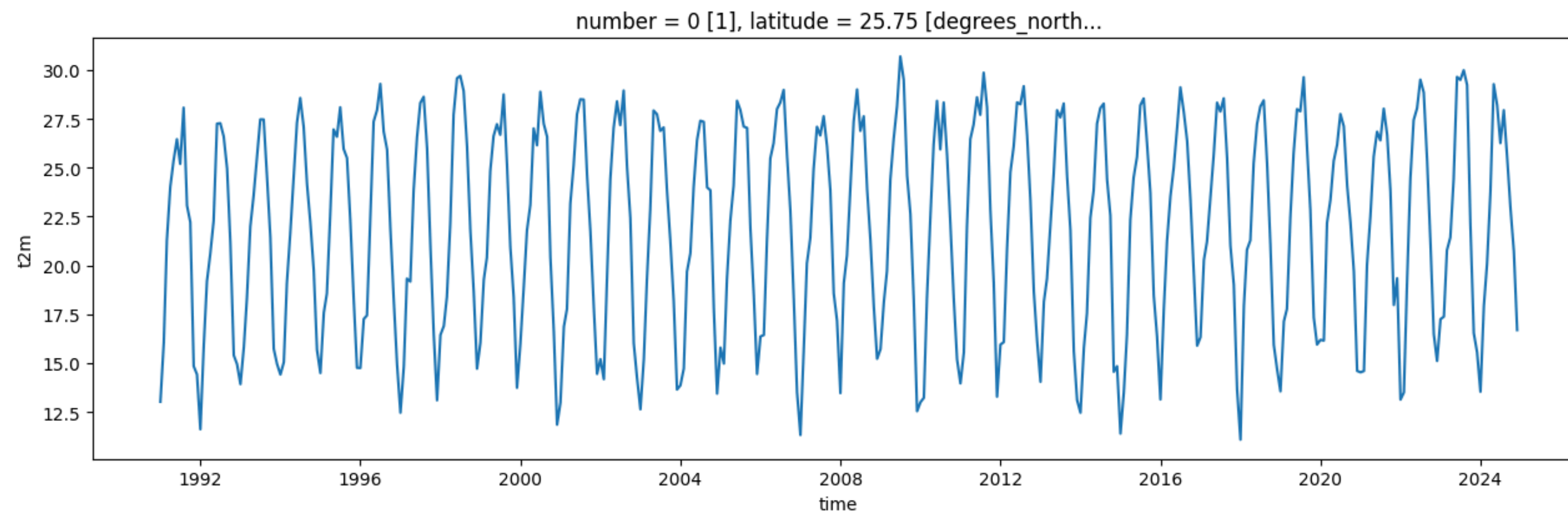
```
import xarray as xr
from pathlib import Path
import matplotlib.pyplot as plt

carpeta_trabajo = Path.home() / 'Desktop' / 'PYTHON_ENES_C2' / 'xarray_practica'
datos = carpeta_trabajo / 'datos' / 'era5_mex'

ds_t = xr.open_dataset(datos / 'data_stream-moda_stepType-avgua.nc')
ds_p = xr.open_dataset(datos / 'data_stream-moda_stepType-avgad.nc')

t_c = ds_t['t2m'] - 273.15
mt_y = t_c.sel(latitude=25.67, longitude=-100.31, method='nearest')

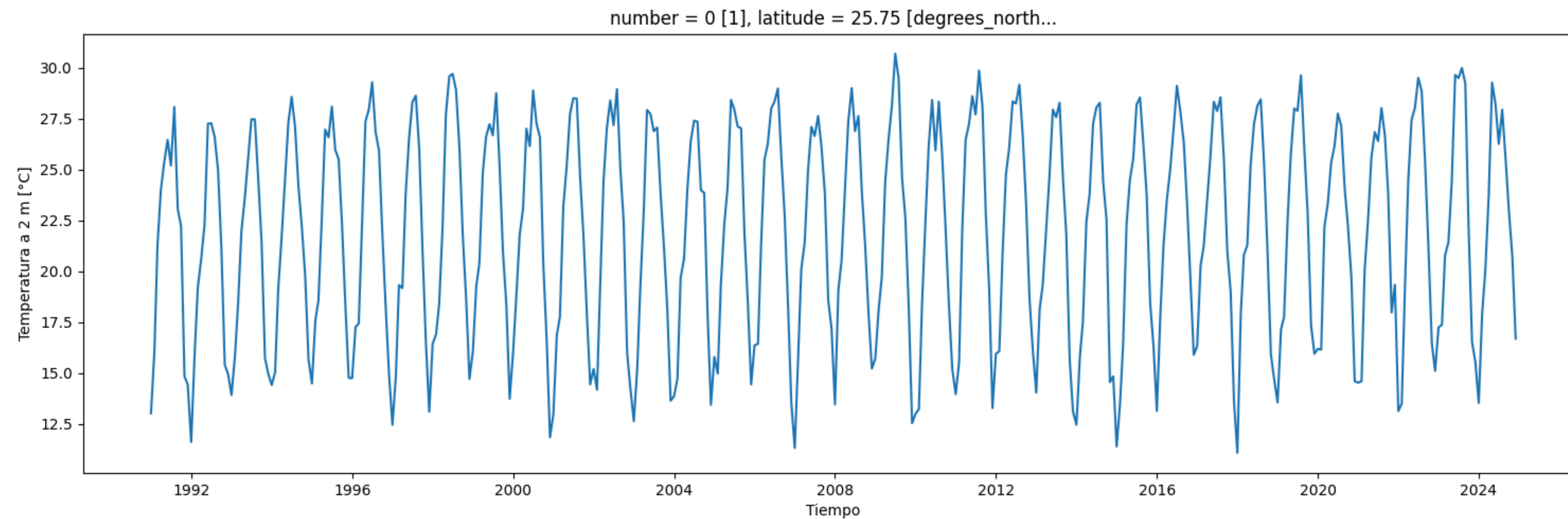
mt_y.plot()
plt.show()
```



Serie de tiempo en un punto

```
mty.attrs['long_name'] = 'Temperatura a 2 m'
mty.attrs['units'] = '°C'
mty['valid_time'].attrs['long_name'] = 'Tiempo'
mty.plot()
plt.show()
```

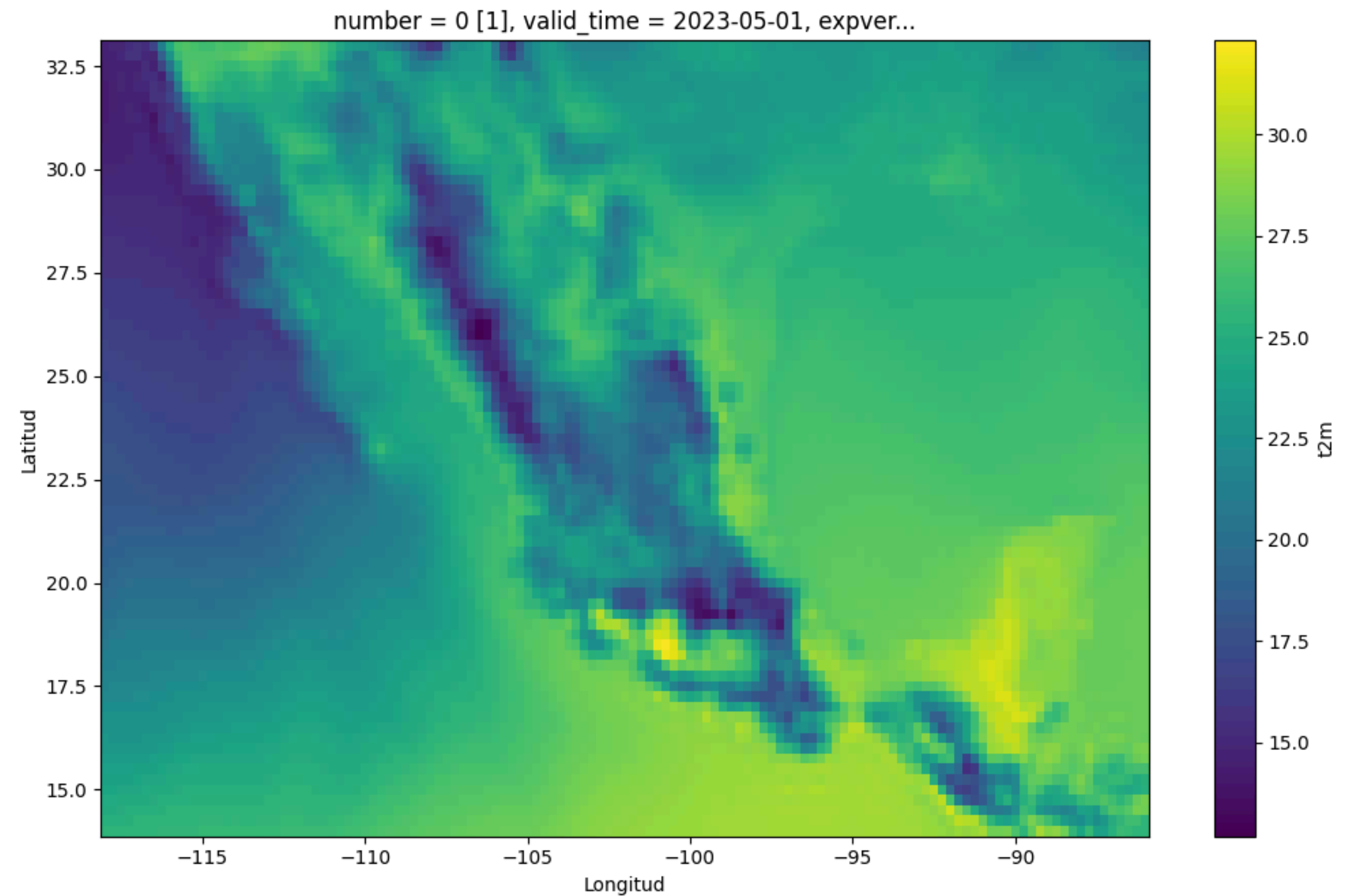
```
mty.plot()
plt.xlabel('Tiempo')
plt.ylabel('Temperatura a 2 m (°C)')
plt.show()
```



Mapa de un mes

```
mayo23 = t_c.sel(valid_time='2023-05-01')
print(mayo23.dims)

mayo23.plot()
plt.xlabel('Longitud')
plt.ylabel('Latitud')
plt.show()
```

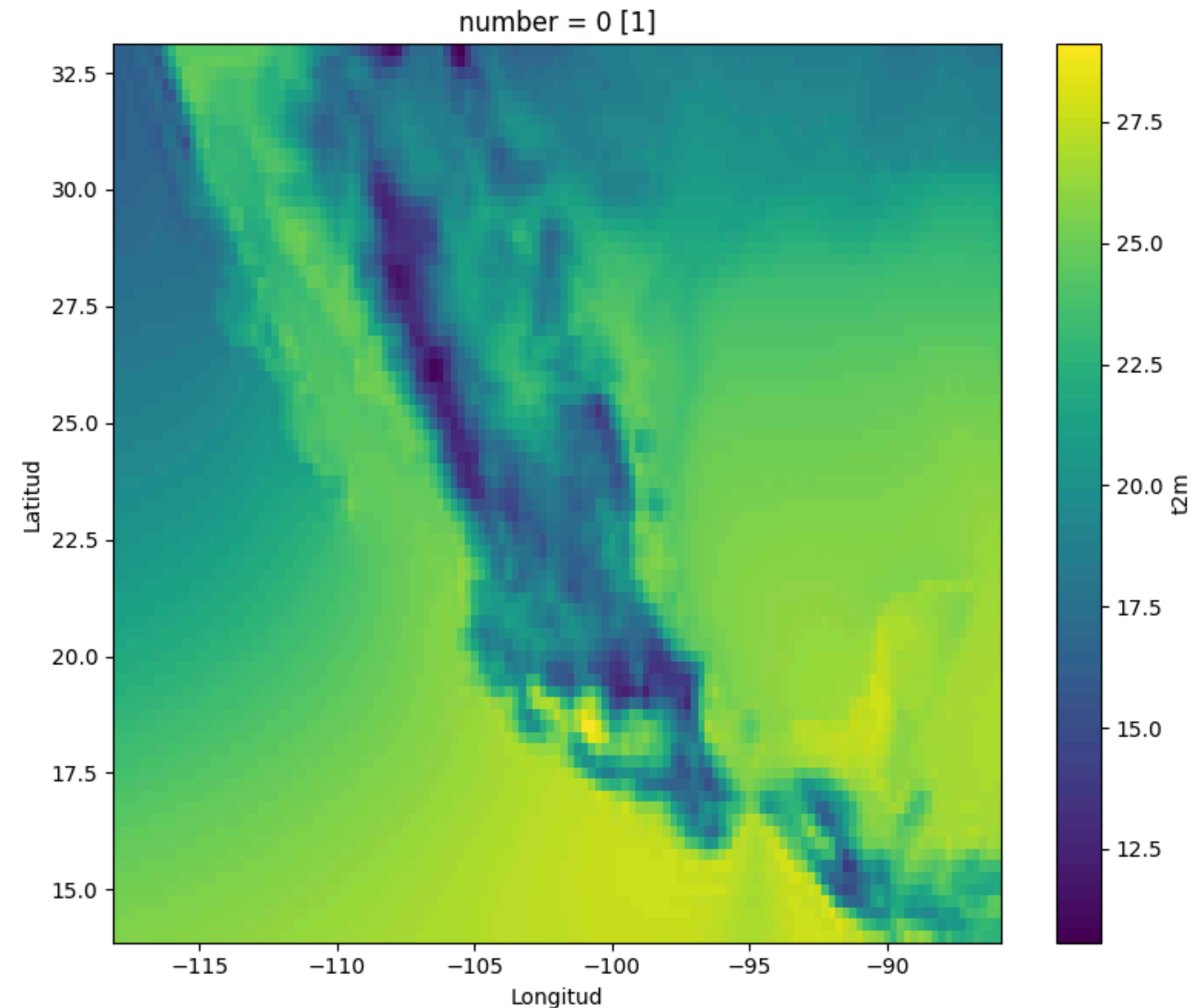


Promedio de todo el periodo 1991-2004

```
t_media = t_c.mean('valid_time')
print(t_media.dims)

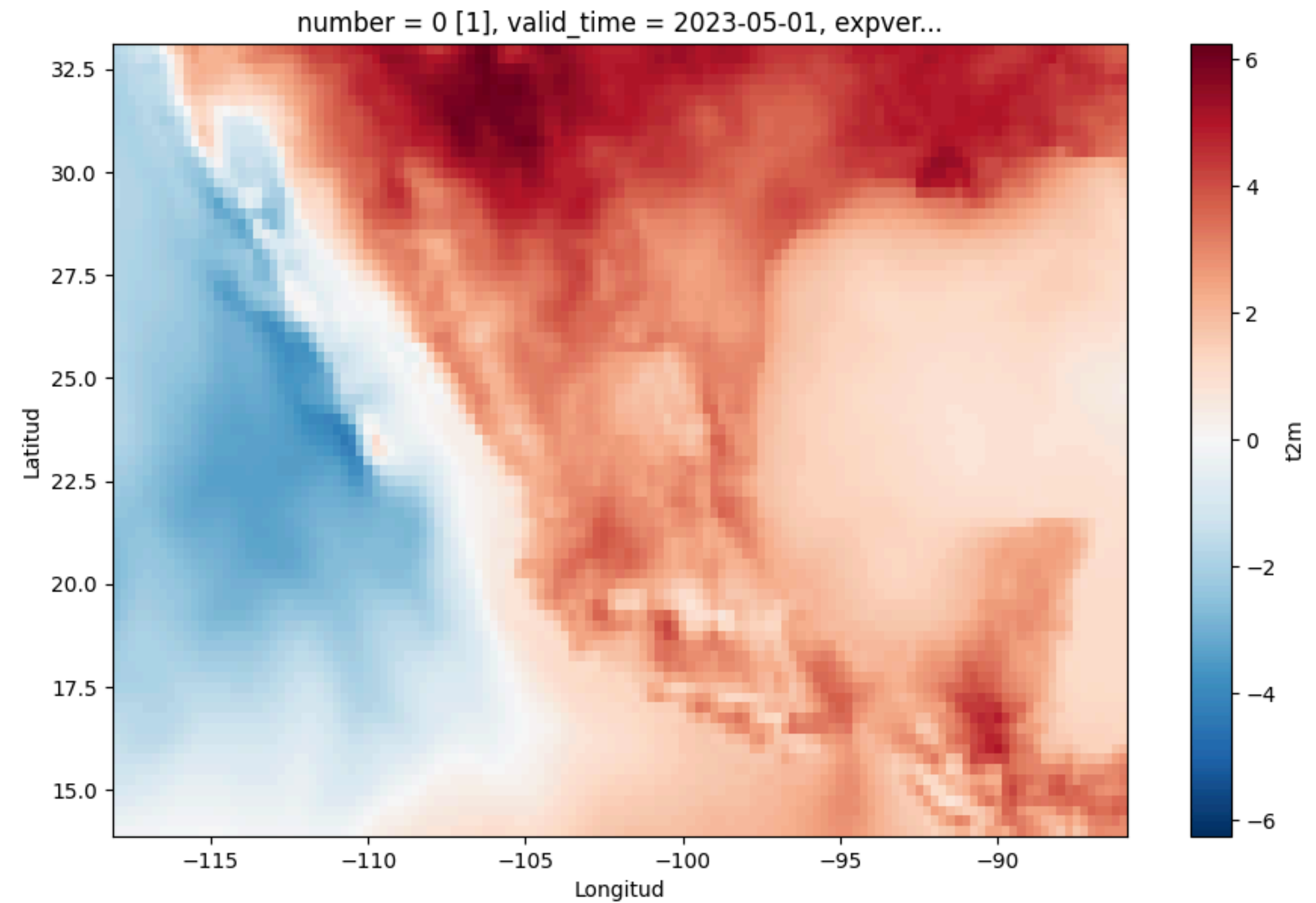
t_media.plot()
plt.xlabel('Longitud')
plt.ylabel('Latitud')
plt.show()
```

*le decimos sobre que
dimensión tiene que calcular el
promedio.



Calcular una diferencia

```
dif = mayo23 - t_media  
dif.plot(cmap='RdBu_r')  
plt.xlabel('Longitud')  
plt.ylabel('Latitud')  
plt.show()
```



Recortar la península de Yucatán

Slice

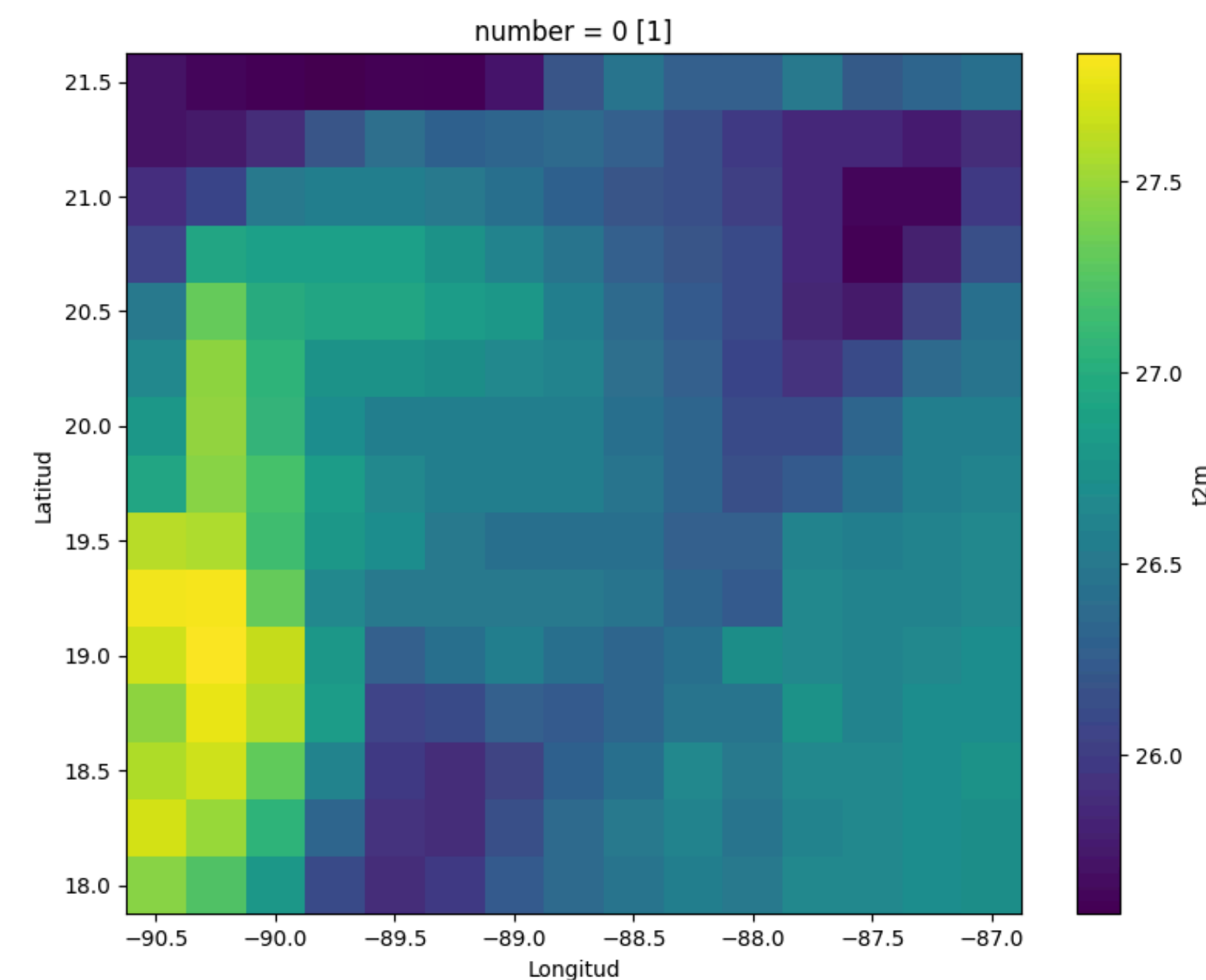
Slice es un objeto de Python (no de xarray ni de numpy) que representa un **rango**: dónde empieza, dónde termina y, opcionalmente, cada cuánto avanza.

```
slice(inicio, fin)
```

```
slice(inicio, fin, paso)
```

```
yuc = t_media.sel(latitude=slice(21.5, 18), longitude=slice(-90.5, -87))
print(yuc.sizes)

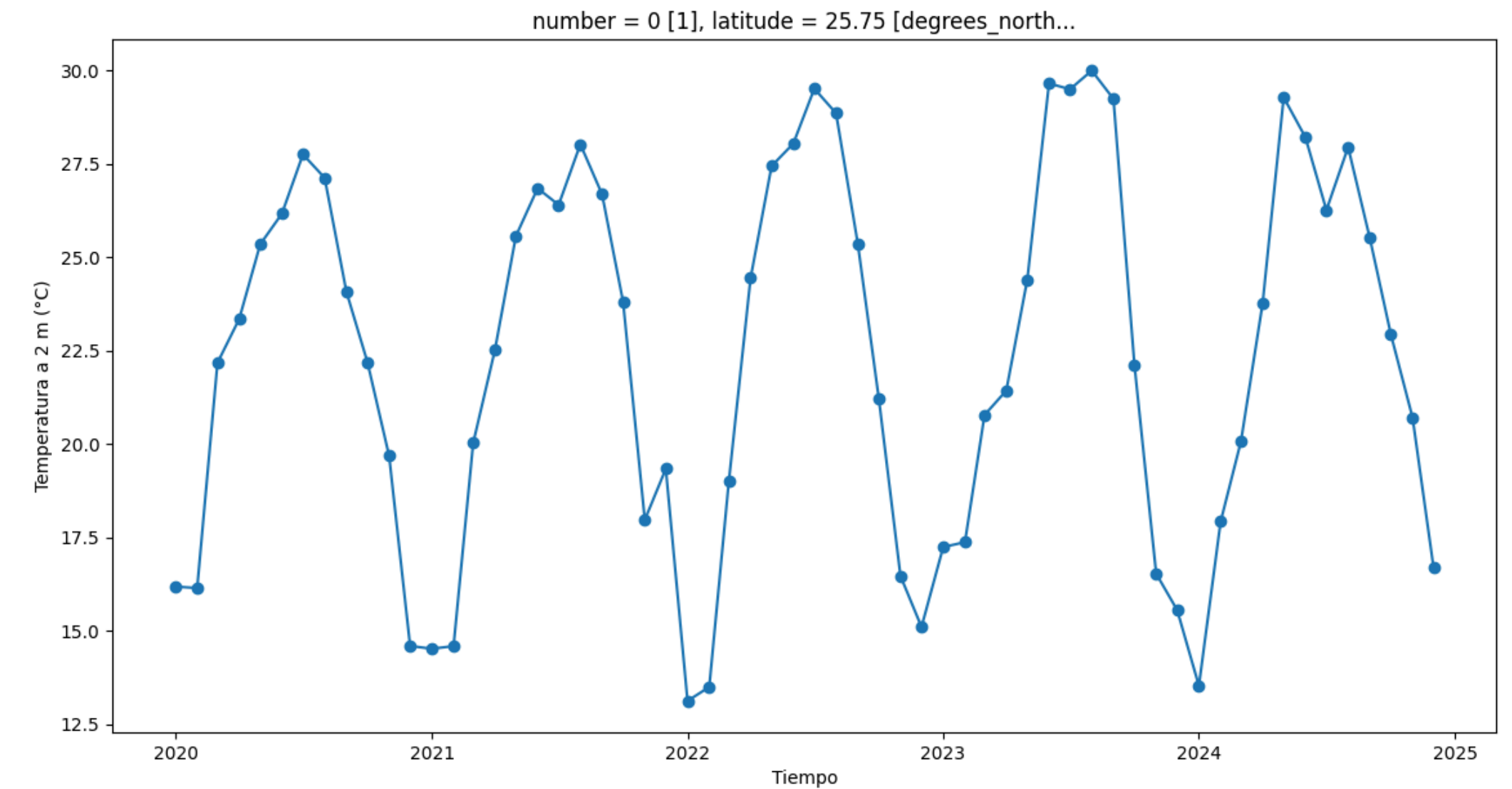
yuc.plot()
plt.xlabel('Longitud')
plt.ylabel('Latitud')
plt.show()
```



Serie de 2020 a 2024

```
mty = t_c.sel(latitude=25.67, longitude=-100.31, method='nearest')
mty_reciente = mty.sel(valid_time=slice('2020', '2024'))
print(mty_reciente.sizes)

mty_reciente.plot(marker='o')
plt.xlabel('Tiempo')
plt.ylabel('Temperatura a 2 m (°C)')
plt.show()
```



groupby

groupby: grupa los datos según una etiqueta, aplica una operación a cada grupo por separado y junta los resultados. Se suele resumir en tres pasos: **separar, aplicar, combinar**.

Por ejemplo, para el promedio de cada mes del año:

1. **Separar:** junta todos los enero en un grupo, todos los febreros en otro, y así hasta diciembre. Son 12 grupos.
2. **Aplicar:** calcula el promedio dentro de cada grupo.
3. **Combinar:** junta los 12 promedios en un solo DataArray.

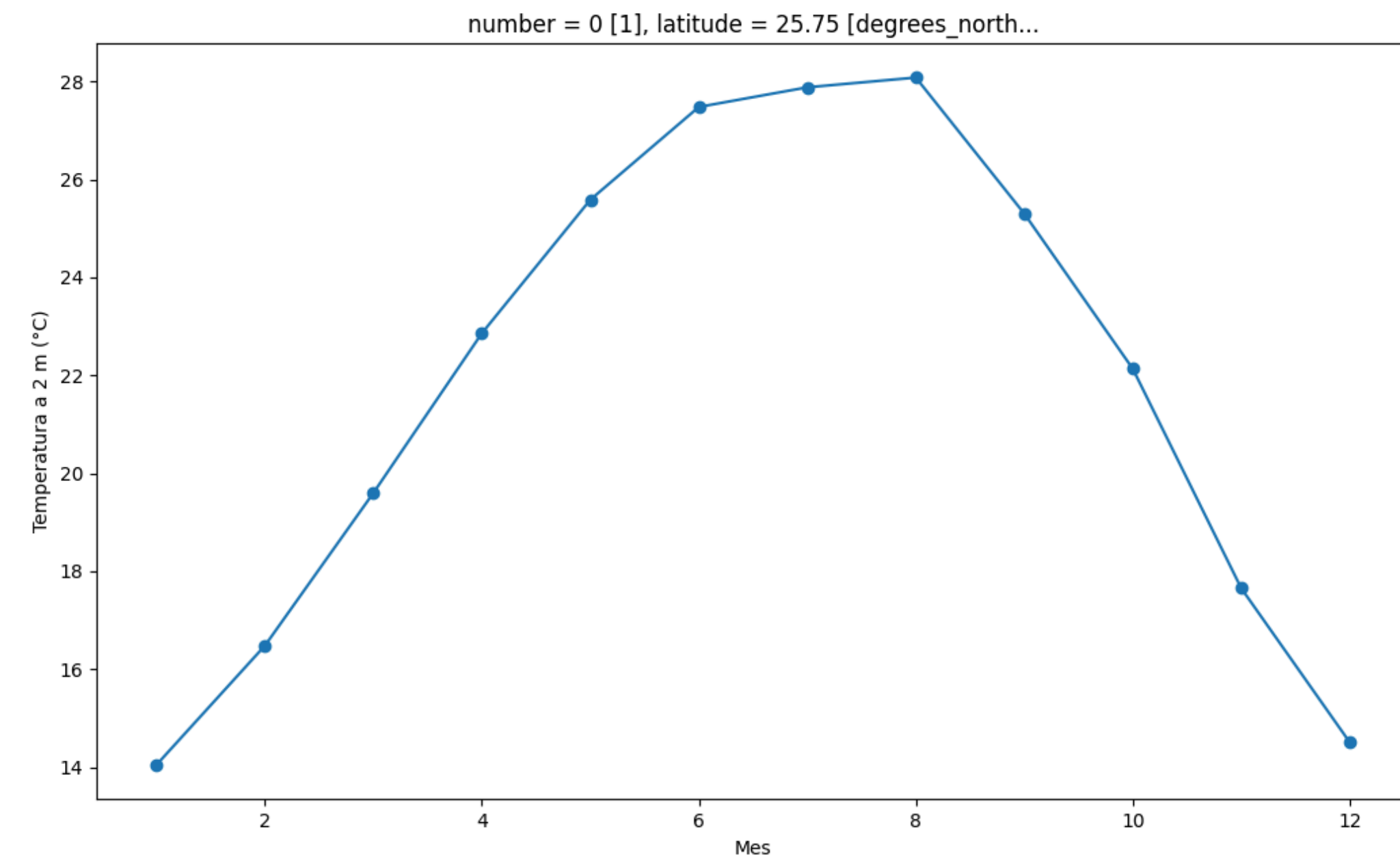
```
print(t_c.groupby('valid_time.month').mean())  
print(mty.groupby('valid_time.month').mean())
```

Ciclo anual

```
mt_ref = mty.sel(valid_time=slice('1991', '2020'))
clim_mty = mty_ref.groupby('valid_time.month').mean()

print(clim_mty)

clim_mty.plot(marker='o')
plt.xlabel('Mes')
plt.ylabel('Temperatura a 2 m (°C)')
plt.show()
```



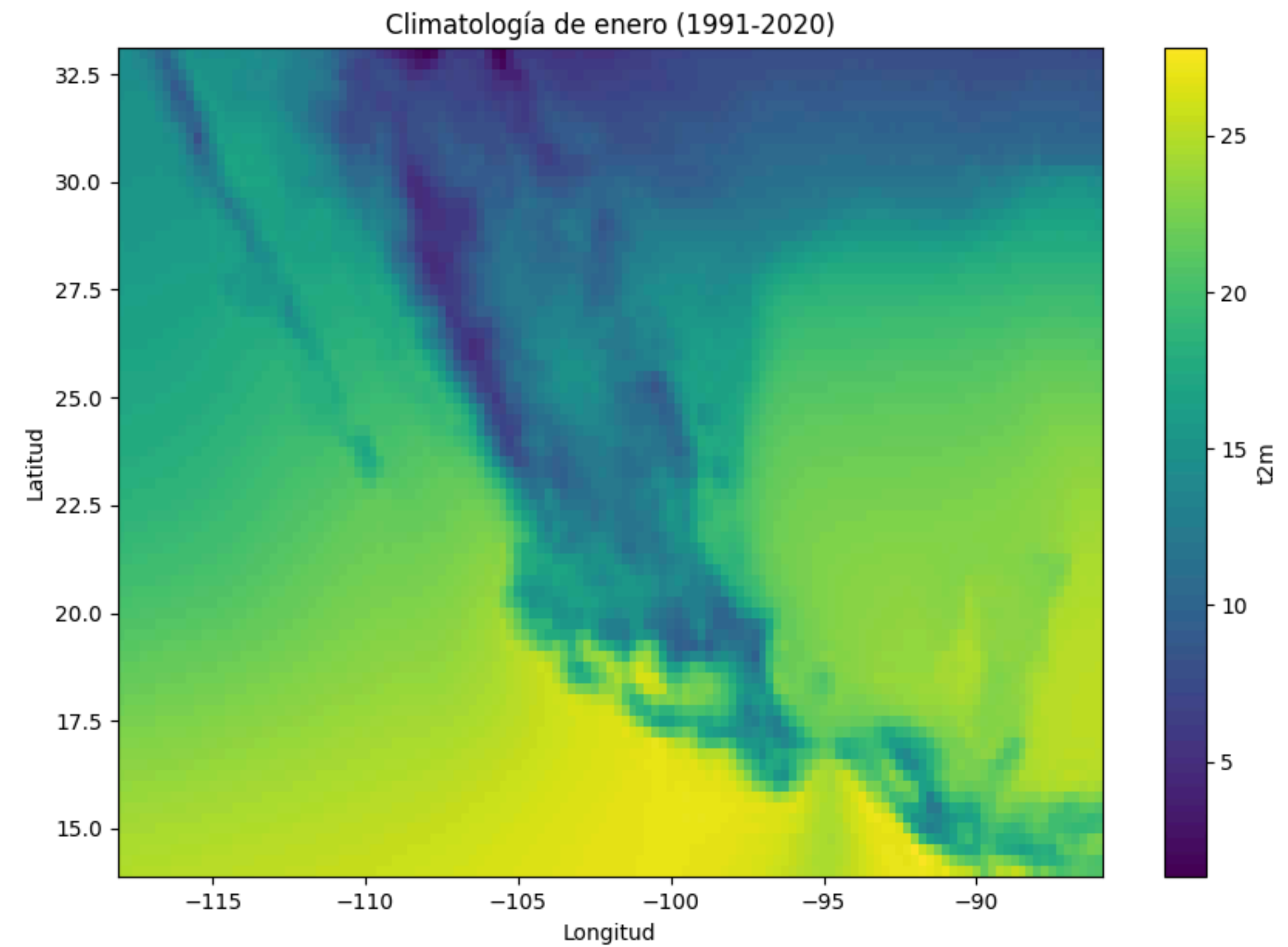
```

t_ref = t_c.sel(valid_time=slice('1991', '2020'))
clim = t_ref.groupby('valid_time.month').mean()

print(clim.dims)

clim.sel(month=1).plot()
plt.title('Climatología de enero (1991-2020)')
plt.xlabel('Longitud')
plt.ylabel('Latitud')
plt.show()

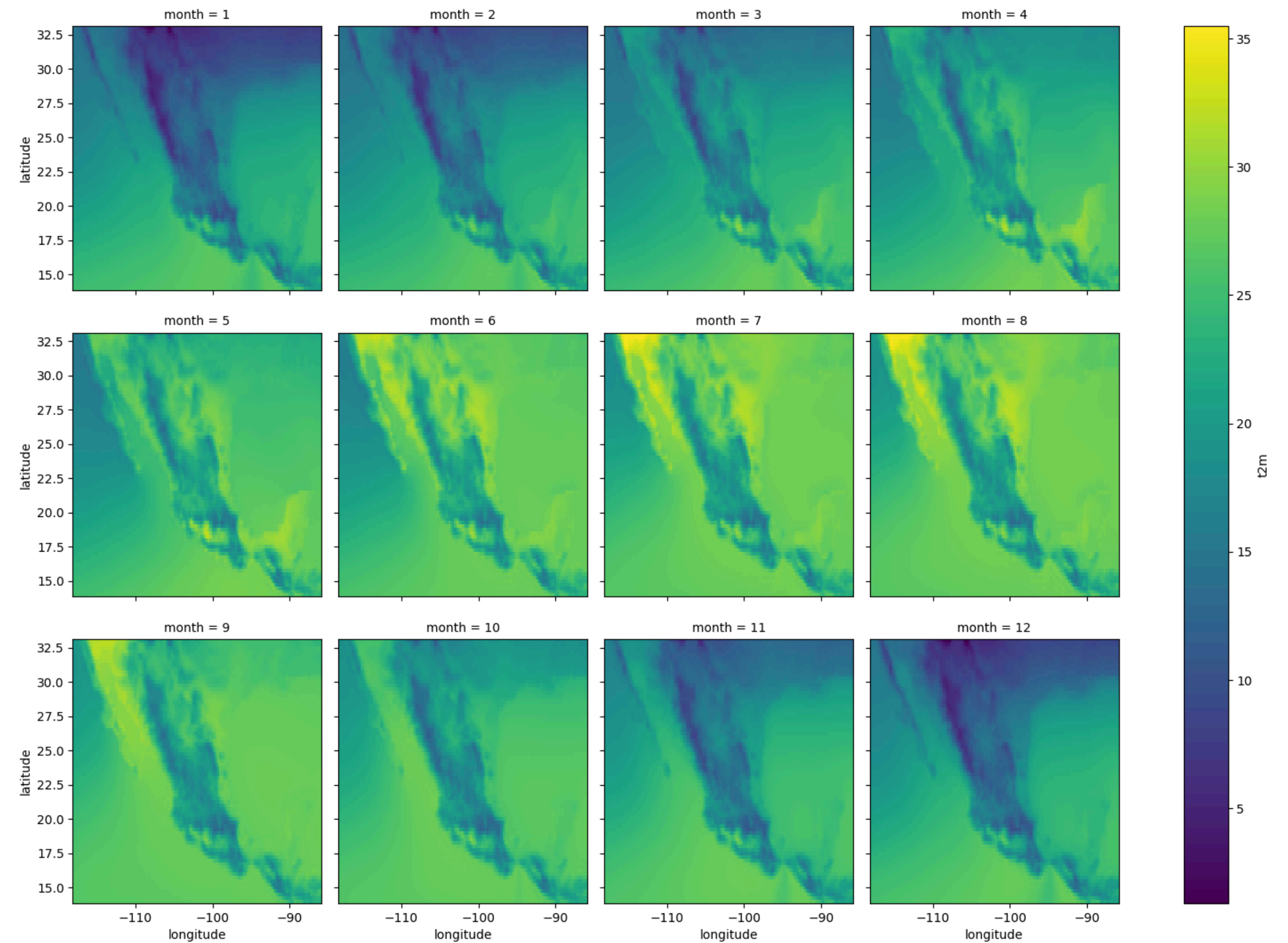
```



```
clim.plot(col='month', col_wrap=4)
plt.show()
```

Indica que ese
haga un mapa
por cada valor
de 'month' en
columnas (uno a
lado de otro)

Indica que
ponga 4 mapas
por fila



Climatología por estación

```
clim_est = t_ref.groupby('valid_time.season').mean()

print(clim_est.dims)
print(clim_est['season'].values)
```

```

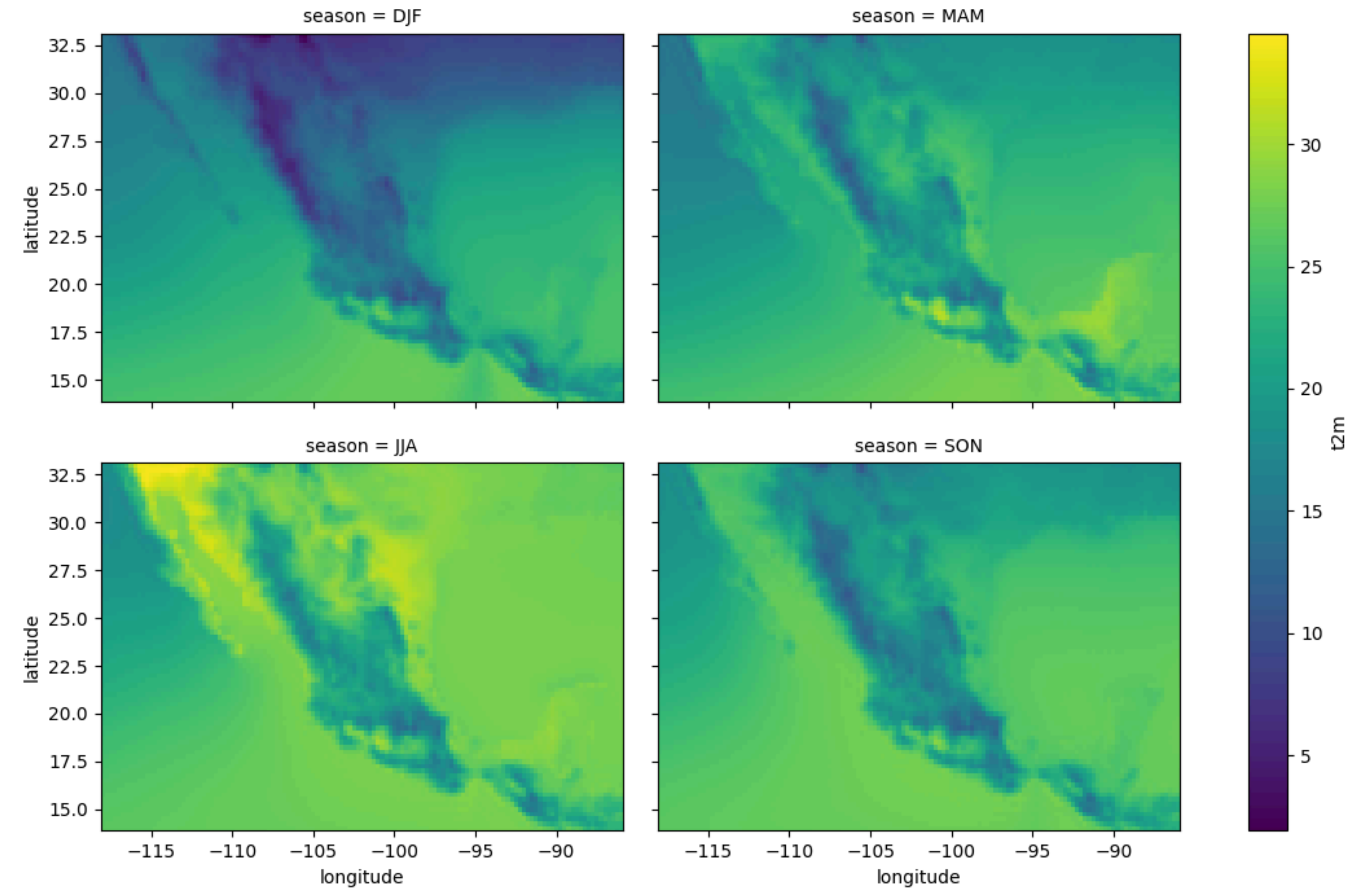
clim_est = t_ref.groupby('valid_time.season').mean()

print(clim_est.dims)
print(clim_est['season'].values)

clim_est = clim_est.sel(season=['DJF', 'MAM', 'JJA', 'SON'])

clim_est.plot(col='season', col_wrap=2)
plt.show()

```



Ejercicio

<https://docs.xarray.dev/en/stable/api/dataarray.html>

- Explorar los archivos el archivo .nc de temperatura
- Hacer lo mismo con la precipitación. Pueden juntar los dos .nc:

```
import xarray as xr
from pathlib import Path
import matplotlib.pyplot as plt

carpeta_trabajo = Path.home() / 'Desktop' / 'PYTHON_ENES_C2' / 'xarray_practica'
datos = carpeta_trabajo / 'datos' / 'era5_mex'

ds_t = xr.open_dataset(datos / 'data_stream-moda_stepType-avgua.nc')
ds_p = xr.open_dataset(datos / 'data_stream-moda_stepType-avgad.nc')

ds = xr.merge([ds_t, ds_p])
```

- Proyecto de precipitación.